

1 Very Busy Expressions

1.1 Formalizzazione del framework

Parametro	Dataflow Problem: Very Busy Expressions
Domain	Set di espressioni $\mathcal{P}(E)$
Direction	Backward $IN[B] = f_B(OUT[B])$ $OUT[B] = \bigwedge_{S \in succ(B)} IN[S]$
Transfer function	$f_B(x) = gen_B \cup (x - kill_B)$
Meet Operation ($\wedge$)	$\cap$
Boundary Condition	$IN[EXIT] = \emptyset$
Initial interior points	$IN[B] = E$

Tabella 1: Parametri del framework per l'analisi Very Busy Expressions.

Le Very Busy Expressions sono espressioni che hanno la garanzia di essere riutilizzate prima di un futuro assegnamento, formalizziamo il framework in questo modo:

- **Dominio:** Definiamo E come tutte le espressioni che hanno un operando non costante, il dominio è l'insieme delle parti di E
- **Direzione Backward:** Prima di poter determinare se un'espressione è "very busy", dobbiamo valutare i suoi utilizzi in futuro, per vedere se viene valutata prima che uno degli operandi sia ridefinito.
- **Meet come Intersezione:** Affinché un'espressione sia considerata "very busy", essa deve essere valutata in tutti i percorsi possibili che partono dal punto corrente verso l'uscita.
- **Condizione a contorno:** Essendo $EXIT$ l'ultimo blocco, non ci sono blocchi futuri CFG analizzato, per questo $IN[EXIT]$ rimane sempre $\emptyset$.
- **Inizializzazione dei punti interni:** Poiché l'operatore di Meet è l'intersezione, i punti interni devono essere inizializzati all'insieme universale per permettere all'algoritmo di eliminare progressivamente le espressioni non valide. Inizializzare a $\emptyset$ renderebbe l'intersezione sempre vuota.

1.2 Struttura del CFG

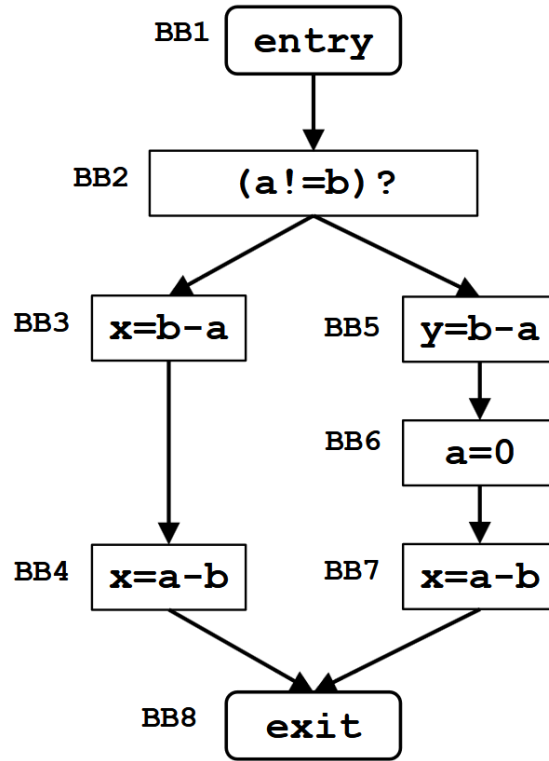


Figura 1: Control Flow Graph del problema analizzato.

Blocco	GEN	KILL
BB1 (entry)	$\emptyset$	$\emptyset$
BB2	$\{b! = a\}$	$\emptyset$
BB3	$\{b - a\}$	$\emptyset$
BB4	$\{a - b\}$	$\emptyset$
BB5	$\{b - a\}$	$\emptyset$
BB6	$\emptyset$	$\{a - b, b - a, a! = b\}$
BB7	$\{a - b\}$	$\emptyset$
BB8 (exit)	$\emptyset$	$\emptyset$

Tabella 2: Insiemi GEN e KILL per il CFG analizzato.

1.3 Tabella delle Iterazioni

Block	Iterazione 0 (Init)		Iterazione 1		Iterazione 2	
	IN[B]	OUT[B]	IN[B]	OUT[B]	IN[B]	OUT[B]
BB1	E	E	$\{b - a, a! = b\}$	$\{b - a, a! = b\}$	$\{b - a, a! = b\}$	$\{b - a, a! = b\}$
BB2	E	E	$\{b - a, a! = b\}$	$\{b - a\}$	$\{b - a, a! = b\}$	$\{b - a, a! = b\}$
BB3	E	E	$\{a - b, b - a\}$	$\{a - b\}$	$\{a - b, b - a\}$	$\{a - b\}$
BB4	E	E	$\{a - b\}$	$\emptyset$	$\{a - b\}$	$\emptyset$
BB5	E	E	$\{b - a\}$	$\emptyset$	$\{b - a\}$	$\emptyset$
BB6	E	E	$\emptyset$	$\{a - b\}$	$\emptyset$	$\{a - b\}$
BB7	E	E	$\{a - b\}$	$\emptyset$	$\{a - b\}$	$\emptyset$
BB8	$\emptyset$	$-$	$\emptyset$	$-$	$\emptyset$	$-$

Tabella 3: Applicazione dell'algoritmo iterativo.

Ad ogni blocco B , le espressioni che sono attualmente "very busy" sono $IN[B]$

2 Dominator Analysis

2.1 Formalizzazione del framework

Parametro	Dominator Analysis
Domain	Set di blocchi $\mathcal{P}(U)$
Direction	Forward $OUT[B] = f_B(IN[B])$ $IN[B] = \bigwedge_{S \in pred(B)} OUT[S]$
Transfer function	$f_B(x) = B \cup x$
Meet Operation ($\wedge$)	$\cap$
Boundary Condition	$OUT[ENTRY] = \{ENTRY\}$
Initial interior points	$OUT[B] = U$

Tabella 4: Parametri del framework per l'analisi Dominator Analysis.

Ogni blocco X è associato a un set $DOM[X]$, a cui appartengono tutti i blocchi che lo dominano. Un blocco Y domina un blocco X se compare in tutti i percorsi tra $ENTRY$ e Y . Formalizziamo il framework in questo modo:

- **Direzione Forward:** Il set $DOM[B]$ è associato al blocco dominato, è necessario effettuare una forward analysis perché il blocco dominante si trova sicuramente prima del blocco dominato.

- **Meet come Intersezione:** Basta un solo percorso tra il blocco *ENTRY* e *X* per invalidare una possibile dominazione su *X*.
- **Condizione a contorno:** *ENTRY* è il nodo iniziale del CFG, Per questo domina se stesso e tutti gli altri nodi.
- **Inizializzazione dei punti interni:** Poiché l'operatore di Meet è l'intersezione, i punti interni devono essere inizializzati all'insieme universale per permettere all'algoritmo di eliminare progressivamente i blocchi non validi.
- **Funzione di Transizione:** La funzione di transizione è semplificata in quanto $gen_B = B$ e $kill_B = \emptyset$.

2.2 Struttura del CFG

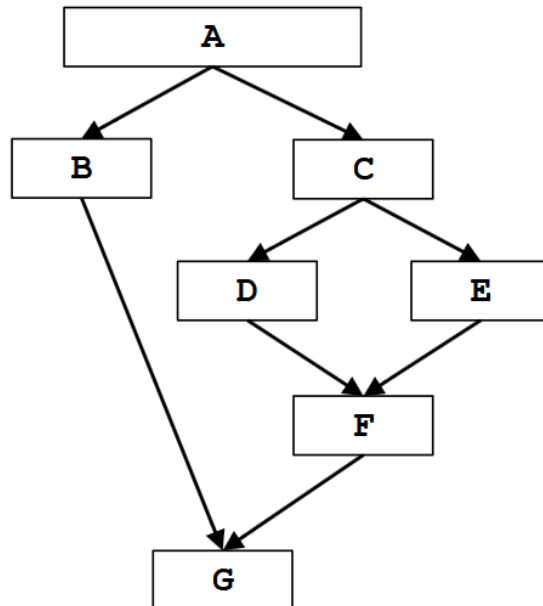


Figura 2: Control Flow Graph del problema analizzato.

2.3 Tabella delle Iterazioni

Block	Iterazione 0 (Init)		Iterazione 1		Iterazione 2	
	IN[B]	OUT[B]	IN[B]	OUT[B]	IN[B]	OUT[B]
A	–	$\{A\}$	–	$\{A\}$	–	$\{A\}$
B	U	U	$\{A\}$	$\{A, B\}$	$\{A\}$	$\{A, B\}$
C	U	U	$\{A\}$	$\{A, C\}$	$\{A\}$	$\{A, C\}$
D	U	U	$\{A, C\}$	$\{A, C, D\}$	$\{A, C\}$	$\{A, C, D\}$
E	U	U	$\{A, C\}$	$\{A, C, E\}$	$\{A, C\}$	$\{A, C, E\}$
F	U	U	$\{A, C\}$	$\{A, C, F\}$	$\{A, C\}$	$\{A, C, F\}$
G	U	U	$\{A\}$	$\{A, G\}$	$\{A\}$	$\{A, G\}$

Tabella 5: Applicazione dell’algoritmo iterativo.

Per ogni blocco B , $DOM[B] = OUT[B]$

3 Constant Propagation

3.1 Formalizzazione del framework

Parametro	Dataflow Problem: Constant Propagation
Domain	Set di coppie $\langle \text{variabile}, \text{costante} \rangle \mathcal{P}(V \times C)$
Direction	Forward $OUT[B] = f_B(IN[B])$ $IN[B] = \bigwedge_{S \in pred(B)} OUT[S]$
Transfer function	$f_B(x) = \text{gen}_B \cup (x - \text{kill}_B)$
Meet Operation ($\wedge$)	$\cap$
Boundary Condition	$OUT[ENTRY] = \emptyset$
Initial interior points	$OUT[B] = V$

Tabella 6: Parametri del framework per l’analisi.

L’obiettivo di quest’analisi è trovare, in ogni nodo del codice, quali variabili hanno un valore costante. Formalizziamo il framework in questo modo:

- **Dominio:** Sia V l’insieme di tutte le variabili e C l’insieme di tutte le costanti. Esiste in ogni nodo un set di coppie $\langle v, c \rangle$ per tutte le variabili costanti all’interno del nodo.

- **Direzione Forward:** Per capire se una variabile è costante, dobbiamo analizzare tutte le sue definizioni precedenti.
- **Meet come Intersezione:** Una variabile è costante se ha lo stesso valore in tutti i nodi precedenti; controlliamo quindi non solo l'esistenza di una coppia sulla variabile v , ma l'uguaglianza tra tutte le coppie $\langle v, c \rangle$
- **Inizializzazione dei punti interni:** Poiché l'operatore di Meet è l'intersezione, i punti interni devono essere inizializzati all'insieme universale, non conoscendo a priori l'insieme delle costanti, inizializziamo all'insieme universo simbolico U , come insieme trasparente rispetto all'intersezione.
- **Funzione di Transizione:** Definiamo kill_B come la ridefinizione di una variabile costante.

Definiamo gen_B come la definizione di una variabile costante quando gli operandi dell'espressione sono costanti o variabili segnate come costanti in precedenza. Se necessario, si completa la generazione di una nuova variabile costante calcolando il valore e aggiungendo una nuova costante all'insieme C .

Gli insiemi gen_B e kill_B sono quindi dinamici e non possono essere determinati a priori, ma vengono generati dall'iterazione dell'algoritmo.

3.2 Struttura del CFG

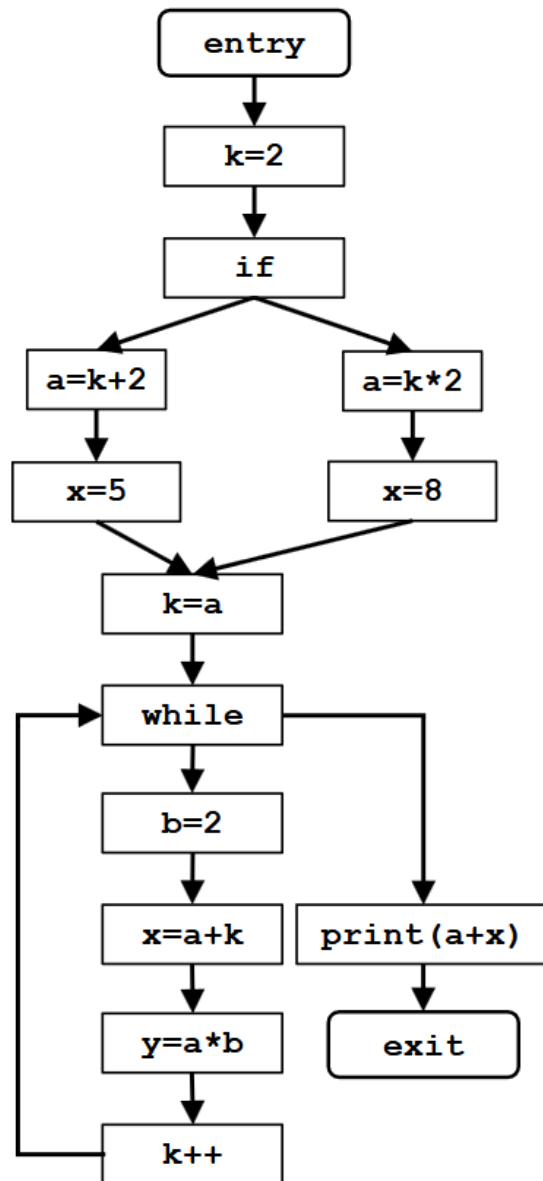


Figura 3: Control Flow Graph del problema analizzato.

3.3 Tabella delle Iterazioni

Block	Iterazione 0 (Init)		Iterazione 1	
	IN[B]	OUT[B]	IN[B]	OUT[B]
BB1 (entry)	—	$\emptyset$	—	$\emptyset$
BB2 (k=2)	U	U	$\emptyset$	$\{\langle k, 2 \rangle\}$
BB3 (if)	U	U	$\{\langle k, 2 \rangle\}$	$\{\langle k, 2 \rangle\}$
BB4 (a=k+2)	U	U	$\{\langle k, 2 \rangle\}$	$\{\langle k, 2 \rangle, \langle a, 4 \rangle\}$
BB5 (x=5)	U	U	$\{\langle k, 2 \rangle, \langle a, 4 \rangle\}$	$\{\langle k, 2 \rangle, \langle a, 4 \rangle, \langle x, 5 \rangle\}$
BB6 (a=k*2)	U	U	$\{\langle k, 2 \rangle\}$	$\{\langle k, 2 \rangle, \langle a, 4 \rangle\}$
BB7 (x=8)	U	U	$\{\langle k, 2 \rangle, \langle a, 4 \rangle\}$	$\{\langle k, 2 \rangle, \langle a, 4 \rangle, \langle x, 8 \rangle\}$
BB8 (k=a)	U	U	$\{\langle k, 2 \rangle, \langle a, 4 \rangle\}$	$\{\langle k, 4 \rangle, \langle a, 4 \rangle\}$
BB9 (while)	U	U	$\{\langle k, 4 \rangle, \langle a, 4 \rangle\}$	$\{\langle k, 4 \rangle, \langle a, 4 \rangle\}$
BB10 (b=2)	U	U	$\{\langle k, 2 \rangle, \langle a, 4 \rangle\}$	$\{\langle k, 4 \rangle, \langle a, 4 \rangle, \langle b, 2 \rangle\}$
BB11 (x=a+k)	U	U	$\{\langle k, 4 \rangle, \langle a, 4 \rangle, \langle b, 2 \rangle\}$	$\{\langle k, 4 \rangle, \langle a, 4 \rangle, \langle b, 2 \rangle, \langle x, 8 \rangle\}$
BB12 (y=a*b)	U	U	$\{\langle k, 4 \rangle, \langle a, 4 \rangle, \langle b, 2 \rangle, \langle x, 8 \rangle\}$	$\{\langle k, 4 \rangle, \langle a, 4 \rangle, \langle b, 2 \rangle, \langle x, 8 \rangle, \langle y, 8 \rangle\}$
BB13 (k++)	U	U	$\{\langle k, 4 \rangle, \langle a, 4 \rangle, \langle b, 2 \rangle, \langle x, 8 \rangle, \langle y, 8 \rangle\}$	$\{\langle k, 5 \rangle, \langle a, 4 \rangle, \langle b, 2 \rangle, \langle x, 8 \rangle, \langle y, 8 \rangle\}$
BB14 (print (a+x))	U	U	$\{\langle k, 4 \rangle, \langle a, 4 \rangle\}$	$\{\langle k, 4 \rangle, \langle a, 4 \rangle\}$
BB15 (exit)	U	U	$\{\langle k, 4 \rangle, \langle a, 4 \rangle\}$	$\{\langle k, 4 \rangle, \langle a, 4 \rangle\}$

Tabella 7: Applicazione dell'algoritmo iterativo.

Block	Iterazione 2	
	IN[B]	OUT[B]
BB1 (entry)	—	$\emptyset$
BB2 (k=2)	$\emptyset$	$\{\langle k, 2 \rangle\}$
BB3 (if)	$\{\langle k, 2 \rangle\}$	$\{\langle k, 2 \rangle\}$
BB4 (a=k+2)	$\{\langle k, 2 \rangle\}$	$\{\langle k, 2 \rangle, \langle a, 4 \rangle\}$
BB5 (x=5)	$\{\langle k, 2 \rangle, \langle a, 4 \rangle\}$	$\{\langle k, 2 \rangle, \langle a, 4 \rangle, \langle x, 5 \rangle\}$
BB6 (a=k*2)	$\{\langle k, 2 \rangle\}$	$\{\langle k, 2 \rangle, \langle a, 4 \rangle\}$
BB7 (x=8)	$\{\langle k, 2 \rangle, \langle a, 4 \rangle\}$	$\{\langle k, 2 \rangle, \langle a, 4 \rangle, \langle x, 8 \rangle\}$
BB8 (k=a)	$\{\langle k, 2 \rangle, \langle a, 4 \rangle\}$	$\{\langle k, 4 \rangle, \langle a, 4 \rangle\}$
BB9 (while)	$\{\langle a, 4 \rangle\}$	$\{\langle a, 4 \rangle\}$
BB10 (b=2)	$\{\langle a, 4 \rangle\}$	$\{\langle a, 4 \rangle, \langle b, 2 \rangle\}$
BB11 (x=a+k)	$\{\langle a, 4 \rangle, \langle b, 2 \rangle\}$	$\{\langle a, 4 \rangle, \langle b, 2 \rangle, \langle x, 8 \rangle\}$
BB12 (y=a*b)	$\{\langle a, 4 \rangle, \langle b, 2 \rangle, \langle x, 8 \rangle\}$	$\{\langle a, 4 \rangle, \langle b, 2 \rangle, \langle x, 8 \rangle, \langle y, 8 \rangle\}$
BB13 (k++)	$\{\langle a, 4 \rangle, \langle b, 2 \rangle, \langle x, 8 \rangle, \langle y, 8 \rangle\}$	$\{\langle a, 4 \rangle, \langle b, 2 \rangle, \langle x, 8 \rangle, \langle y, 8 \rangle\}$
BB14 (print (a+x))	$\{\langle a, 4 \rangle\}$	$\{\langle a, 4 \rangle\}$
BB15 (exit)	$\{\langle a, 4 \rangle\}$	$\{\langle a, 4 \rangle\}$

Tabella 8: CONTINUO: Applicazione dell'algoritmo iterativo.

Le variabili costanti sono rappresentate in ogni blocco B dal set $OUT[B]$. Si può notare come la variabile k , che aumenta ad ogni ciclo, sia stata rimossa dalle costanti alla seconda iterazione del programma.